PROJECT REPORT

roomcraft

An AI interior designer: scan a real room with a phone, design it in 2D and 3D, and furnish it with an AI assistant



Team members	[Team member 1, Roll no.] [Team member 2, Roll no.] [Team member 3, Roll no.] [Team member 4, Roll no.]
Guide	[Guide name, designation]
Department	[Department]
Institution	[College / University]
Academic year	[2026–27]
Date	November 2026

Abstract

Planning a room usually means a tape measure, graph paper and a lot of guessing about whether the furniture will fit. roomcraft is a web application that removes those steps. A user can measure a real room with an Android phone's camera in the browser, open it as an editable floor plan and 3D model, and furnish it by hand or by asking an AI assistant in plain English. Every design is saved to the user's account and can be exported as a labelled PDF floor plan.

The system has four parts. A **React + three.js frontend** provides the dashboard, the 2D floor plan and the 3D room editor. An **Express + PostgreSQL backend** handles accounts (email OTP sign-up, JWT login), profiles and saved designs. A **WebXR room scanner** measures wall positions, ceiling height, doors, windows and furniture with the phone's AR tracking. An **AI layer** built on Google Gemini turns plain-English requests into furnished rooms and suggestions, while the app itself places every piece with deterministic collision rules, so the AI never produces overlapping or out-of-bounds furniture.

The scanner measured a ceiling at 2.72 m against a tape reading of 2.74 m, and straight lines on an open floor to within 1–2 cm. Its geometry is covered by 306 automated tests. The editor supports 14 room types, 86 furniture types, any furniture angle, connected rooms, and works on phones as well as laptops.

Contents

1 Introduction

2 System overview and architecture

3 Technology stack

4 Accounts, profile and dashboard

5 Room editor: 2D plan and 3D view

6 AR room scanner

7 AI assistant and camera detection

8 Floor plan export

9 Data model and API

10 User interface: themes, phones, tutorial

11 Security

12 Testing and results

13 Limitations and future work

14 Conclusion

References

1. Introduction

1.1 Problem

Anyone furnishing a room faces the same questions: how big is the room really, will this bed fit next to the wardrobe, does the door still open, and what will it look like? Professional design tools answer these but expect exact measurements and hours of learning. Simple room planners are easy to use but start from an empty box that the user has to measure and draw first, and they leave the design decisions to the user.

1.2 Objectives

1. Measure a real room (size, ceiling height, doors, windows, furniture) with a phone, without installing an app.
2. Edit the room as a 2D floor plan and a 3D model, with furniture that cannot overlap, leave the room or block a door.
3. Let users describe a room or a change in plain English and apply AI suggestions with one tap, and undo them.
4. Keep every design in the user's account, and export it as a labelled PDF floor plan.
5. Work equally well on a phone and a laptop, in light and dark themes.

1.3 Scope

roomcraft covers residential and institutional rooms (14 types, from bedroom and kitchen to classroom and computer lab), rectangular rooms joined through doors into multi-room layouts, and a catalogue of 86 furniture types in four styles. The scanner runs on Android phones with ARCore in Chrome; iPhones are out of scope because Safari does not offer WebXR augmented reality.

1.4 Report structure

Chapter 2 gives the architecture and chapter 3 the technologies. Chapters 4 to 8 describe each module in turn. Chapters 9 to 11 cover the data model, user interface and security, chapter 12 the testing and results, and chapters 13 and 14 the limitations, future work and conclusion.

2. System overview and architecture

roomcraft is a client–server web application. The browser runs two pages from the same frontend: the main React app (`index.html`) and the AR room scanner (`scan.html`). Both talk only to the frontend's own address; API calls under `/api` and uploaded files under `/uploads` are passed on to the Express backend, which stores everything in PostgreSQL. Calls to Google Gemini are made from the server side, so the API key never reaches the browser.

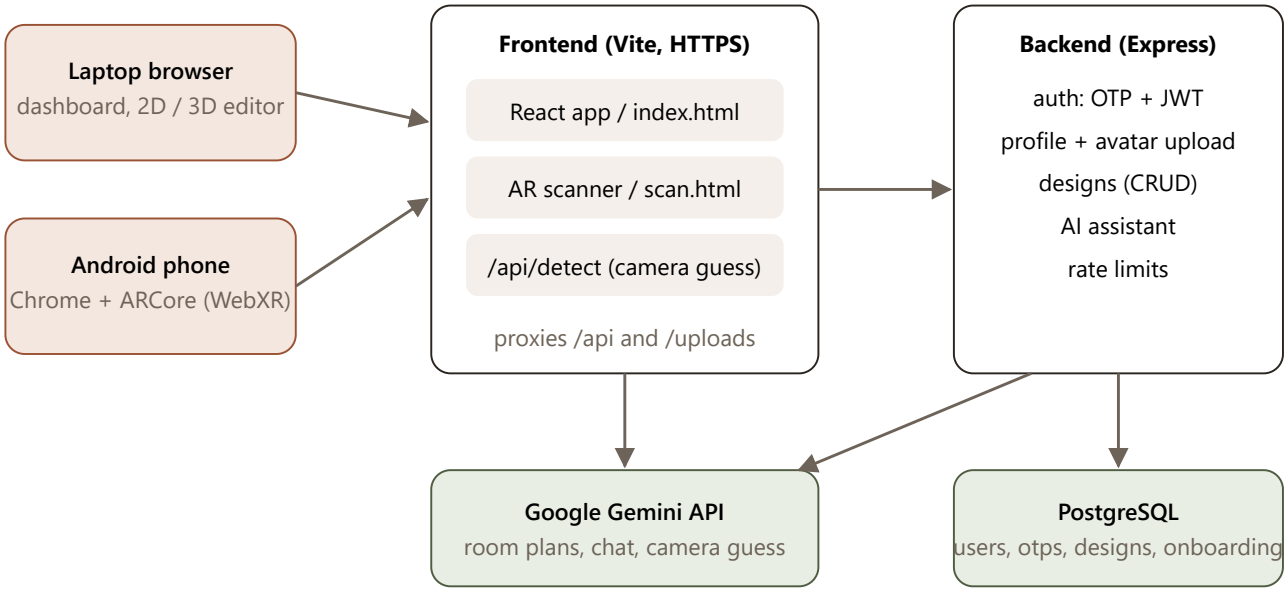


Figure 2.1: How the parts of roomcraft talk to each other

2.1 A typical session

1. The user signs up with an emailed 5-digit code and answers a few onboarding questions.
2. On the phone, **Scan my room** opens the scanner. The user taps the corners of each wall, then the doors, windows and furniture, and saves the room to their account.
3. On the laptop the scanned room appears in **My Designs**. The editor shows it as a 2D plan and a 3D model with its real size, ceiling height, doors and furniture.
4. The user rearranges furniture, changes colours and floors, adds connected rooms, or asks the AI assistant ("furnish this as a cozy study") and applies its suggestions.
5. The design is saved and can be exported as a PDF floor plan.

2.2 Design principles

- **The app places, the AI chooses.** The language model only picks room types, furniture and colours from the app's own catalogue; positions always come from the app's placement rules.
- **One source of truth for geometry.** All lengths are in feet, the room is centred on the origin, and the same collision rules are used by the editor, the AI placer and the scanner's tidy-up step.
- **Keys stay on the server.** The database password, JWT secret and Gemini key live only in server-side environment files.

3. Technology stack

Layer	Technology	Used for
Frontend framework	React 19, React Router, Vite 8	Pages, routing, dev server with HTTPS for phones, production build
Styling	Tailwind CSS v4	Light and dark themes from shared colour tokens, responsive layouts
3D graphics	three.js, @react-three/fiber, drei	3D room, furniture models, orbit camera, dragging pieces in 3D
2D plan	SVG rendered by React	Scalable floor plan, labels, size lines; also reused for PDF export
PDF	jsPDF	A4 floor-plan export
Backend	Node.js, Express	REST API, file uploads (multer), rate limiting
Database	PostgreSQL 16 (Docker locally)	Users, one-time codes, onboarding answers, designs (JSON columns)
Authentication	JWT, bcrypt, email OTP (Resend)	Sign-up verification, login, password reset
AR scanning	WebXR Device API (immersive-ar, hit-test, anchors, DOM overlay)	Measuring the room with the phone camera in Chrome
On-device vision	MediaPipe Tasks Vision (EfficientDet-Lite0)	Spotting furniture in the camera view during a scan
Generative AI	Google Gemini (flash-lite, flash) with JSON schemas	Chat suggestions, furnishing a room from a request, naming a piece from a photo
Testing and quality	Vitest, oxlint	306 scanner tests; lint across the frontend

~19,000
lines of code

19
API routes

4
database tables

2
web pages: app and scanner

Code size counts the frontend (about 11,700 lines), the backend (about 1,900) and the scanner (about 5,300), excluding tests and third-party libraries.

4. Accounts, profile and dashboard

4.1 Sign-up and login

Registration asks for a name, email and password. The backend stores a bcrypt hash of the password and emails a 5-digit one-time code; the account becomes active only after the code is entered. Login returns a JSON Web Token that the frontend keeps in local storage and attaches to every request. A request that comes back as unauthorised returns the user to the login page. Forgotten passwords use the same code-by-email flow.

4.2 Onboarding and profile

First-time users answer short onboarding questions. The profile page lets them change their name, profession and password and upload a profile photo, which is stored on the server and shown in a circle in the dashboard's top bar.

4.3 Dashboard

My Designs lists the saved designs as cards with the room type, size and last update. From here the user opens a design, starts a new one, exports a design's floor plan, deletes one, or starts a phone scan with **Scan my room**. On a phone the top bar shortens to icons and the action buttons sit side by side.

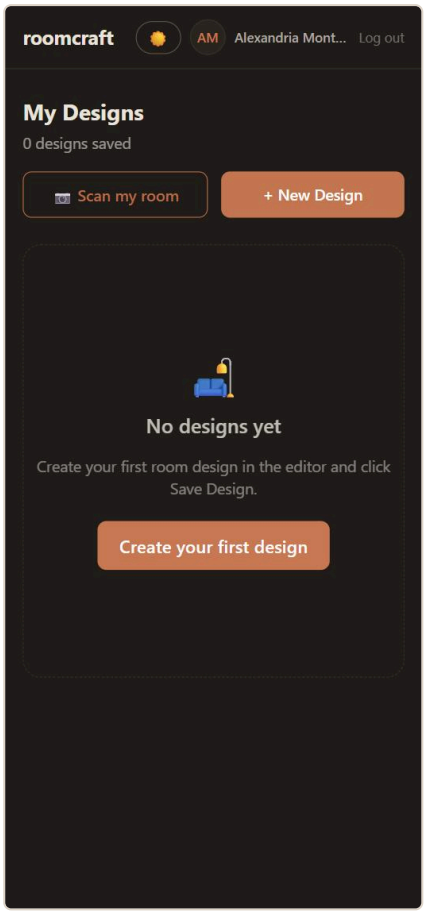


Figure 4.1: Dashboard on a phone (test account)

5. Room editor: 2D plan and 3D view

The editor is where a room is built and furnished. A room panel on the left sets the room type, width, length, height and style; **Generate room** fills it with a typical furniture set for that room type, placed by the app's rules. The same room can be viewed as a flat 2D plan or a 3D model, and every change in one shows in the other.



Figure 5.1: 3D view of a generated 14 × 16 ft living room

5.1 Working with furniture

- **Move** by dragging in either view. A piece that would hit another piece, a wall or a door's swing stops at the last free position instead of overlapping.
- **Rotate** to any angle with a slider or 15° steps; the piece snaps straight near 0°, 90°, 180° and 270°, and an angle that would cause a collision is refused.
- **Resize** width, depth and height to any number of decimals.
- **Doors and windows** slide along their wall, change size, and doors swing open, flip hinge and open in or out.
- **Look**: wall colours per wall (with AI-suggested palettes), seven floor materials, four styles.

5.2 Placement rules

All placement goes through one set of rules (`utils/layout.js`), used by the editor, the room generator, the AI placer and the scanner's tidy-up:

Rule	What it does
Inside the room	Every piece's rotated footprint must stay inside the inner faces of the walls.
No overlap	Two pieces keep at least 0.05 ft apart (rotated rectangles are compared exactly).
Wall clearance	A piece against a wall sits 0.13 ft from the wall's centre line, flush with its inner face.
Door swing kept clear	Each door has a keep-out area the size of its swing; no floor piece may enter it.
Openings kept clear	Wall-mounted pieces (TV, painting, shelf) cannot cover a door or window, doors and windows cannot overlap each other, and a window AC must sit inside a window frame.
Free-spot search	When a new piece is added, the app searches along the walls and then the floor for the nearest legal spot.

5.3 The 2D floor plan

The plan is drawn as SVG, so it stays sharp at any zoom. Every piece is labelled inside its own box. Labels run along a piece's long side, use one or two lines, and slide or shrink so that no two labels touch, including when a wall item such as a cabinet hangs over a washing machine. Every room carries its width and length as size lines on a side with no neighbouring room. Zoom buttons, a fit-to-room button and pinch or scroll zoom are provided.



Figure 5.2: Bathroom: every piece labelled



Figure 5.3: Kitchen: cabinets, appliances, door swing

5.4 Connected rooms

A **+** box outside each door adds a second room through that door. Rooms dock to each other wall to wall, can be dragged along the shared wall, turned in 90° steps, or the whole layout turned at once. The

doorway between two rooms is drawn open on both sides, and each room keeps its own furniture, colours and floor.



Figure 5.4: Four connected rooms; long names wrap inside their room and each room has its own size lines

6. AR room scanner

The scanner measures a real room with the phone's camera, in the browser, with no app to install. It uses the WebXR Device API in Chrome on Android (ARCore): the phone tracks its own position in the room, and a *hit test* returns the 3D point where the centre of the screen meets the floor or a wall.

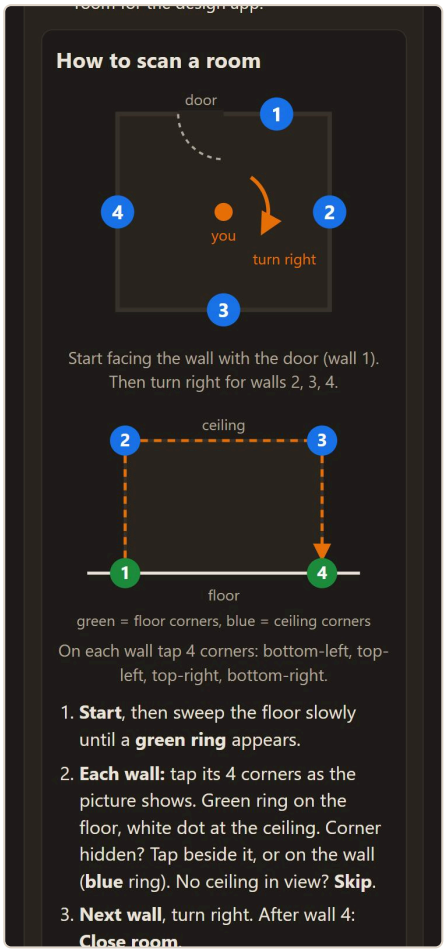


Figure 6.1: The scanner's picture guide

6.1 Scanning steps

1. **Walls.** Facing the wall with the door, the user taps four corners in a U: bottom-left on the floor, top-left and top-right at the ceiling, bottom-right. Then they turn right to the next wall.
2. **Close room.** After the fourth wall the app offers to close the room when the last corner meets the first.
3. **Doors and windows** are captured by two corners; **furniture** by its type and size and one tap. Tall cupboards take two corners, like a door.
4. **Save to my designs** sends the room to the user's account, ready to open in the editor.

A small diagram on screen shows the current wall and which corner is next, and a short hint at the start points to where progress is shown.

6.2 How the room is computed

- **Walls as lines.** Each wall is a line on the floor through its bottom-corner taps. The room is the rectangle that best fits all four lines: its angle from the four wall directions, its width and length from the distance between opposite walls. Exact corner points are never needed, so a corner hidden behind furniture does not spoil the result.
- **Furniture in the way.** Taps more than 20 cm above the floor are left out of the wall fit; the wall is then placed from its top corners, which furniture never hides.
- **Ceiling height** comes from where the aiming lines through the top-corner taps meet each wall's plane.
- **Doors and windows**: two aimed corners meet the wall's plane, giving position, width, height and sill.
- **Tidy-up**: before saving, every floor piece is turned square to the walls and moved out of any overlap, using the same rules as the editor.

Every result is recomputed from the raw taps, which are saved in a scan log. An old scan therefore improves automatically when the geometry code improves, and a saved log can be reopened on a laptop with **Open a scan file**.

7. AI assistant and camera detection

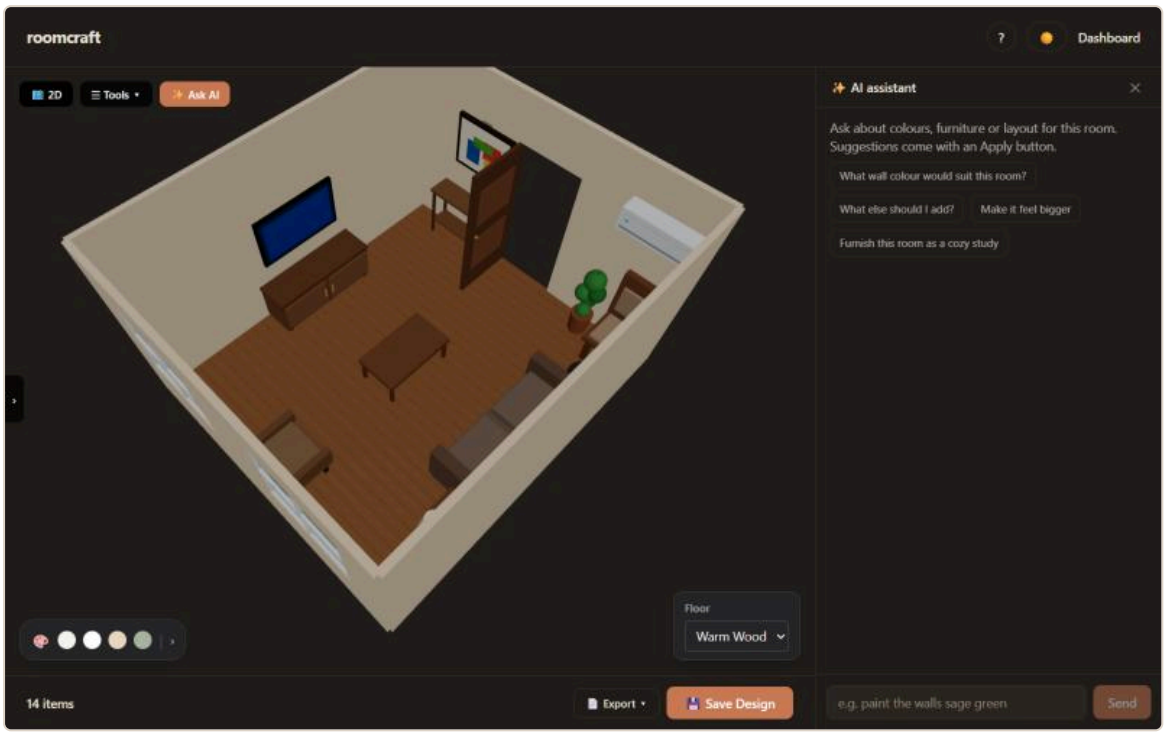


Figure 7.1: The ✨ Ask AI panel opens as a column beside the room on a laptop

7.1 What the assistant does

✨ **Ask AI** in the editor answers questions about the open room and proposes changes: paint walls, change the floor, add or remove a piece, or furnish the whole room ("furnish this as a cozy study"). Every concrete suggestion appears as a button. Nothing changes until the user taps it, and every applied change can be undone. "Furnish this room as…" keeps the room's size, doors and windows, so it works on a scanned room too: scan the room, then ask the assistant to furnish it.

7.2 How it is kept reliable

Technique	Effect
Catalogue sent with every request	The model can only choose furniture, styles, floors and paints that the app can draw.
Structured output (JSON schema)	Answers come back as typed actions (`set_wall_color` , `add_furniture` , `furnish_room` …), never free-form coordinates.
Server-side checking	Every id is checked against the catalogue again; paint names are repaired to valid colours; a reply that promises a change but carries no action is retried.
App does the placing	Pieces are placed by `utils/promptRoom.js` with the editor's rules, so AI-made rooms never overlap or block doors.
Time limits and fallback	40 s per attempt, 75 s in total, with a faster model first and a fallback model if it is busy.
Rate limit	12 assistant requests per minute per user.

7.3 Camera detection during a scan

When adding furniture during a scan, the user can tap **Guess from camera** instead of picking the type. The phone first makes a quick guess on the device with MediaPipe's EfficientDet-Lite0 object detector (beds, chairs, sofas, tables, TVs and similar), then sends one camera frame to Gemini through the server, which answers with a furniture type from the app's catalogue. The best match is highlighted in the furniture picker; the user taps it to confirm or picks another type, so a wrong guess never lands in the room unnoticed.

8. Floor plan export

Any design can be exported as an A4 PDF from the editor or the dashboard. The export draws the same React component as the on-screen 2D plan, off screen, then converts it to an image and places it on the page with the design's name, room type, size, number of items and date. The PDF therefore always matches what the user sees: every room with its size lines, every piece labelled inside its box, doors with their swings and windows. On a phone the PDF can be shared directly through the phone's share sheet.



Figure 8.1: Bedroom plan as exported



Figure 8.2: Dining room plan as exported

9. Data model and API

9.1 Database tables

Table	Contents
`users`	Name, email, bcrypt password hash, profession, profile photo path, verified and onboarding flags.
`otps`	One-time codes for sign-up and password reset; each has an expiry time and works only once.
`onboarding_responses`	Answers to the first-time questions.
`designs`	One row per design: name, room type, width, length and wall height in feet (4 decimals), style, floor, and JSON lists of furniture, doors and windows, wall colours and connected rooms.

Storing furniture and openings as JSON keeps a whole design in one row, so opening or saving a design is a single query, and new furniture fields can be added without a database migration.

9.2 REST API

Group	Routes	Purpose
Auth	4	Register, log in, forgot and reset password
OTP	3	Send, verify and resend one-time codes
Profile 🔒	4	Read and update profile, change password, upload photo
Onboarding 🔒	1	Save the first-time answers
Designs 🔒	5	List, read, create, update, delete designs
Assistant 🔒	2	Plan a room from text; chat about the open room

🔒 needs a valid login token.

9.3 Coordinates

All lengths are in feet. The room is centred on the origin with the front wall at −z. A floor piece stores its centre (`x`, `z`), size and rotation; a wall piece stores its wall, its position along that wall and its height above the floor. Connected rooms store their offset from the main room.

10. User interface: themes, phones, tutorial

- **Light and dark themes** come from one set of colour tokens and are remembered per device; the scanner page follows the same theme.
- **Phones.** The editor fills the screen with the top bar and bottom bar pinned. Tools sit in a **☰ Tools** menu, and the selected piece's size, rotation and delete controls appear next to it. Panels become bottom sheets, and no control overlaps another at phone width.
- **Guided tour.** The **?** button starts an 8-step tour that spotlights each control. On a phone the card moves to the top or bottom of the screen, away from what it points at.
- **Short instructions** everywhere, with pictures in the scanner instead of long text.

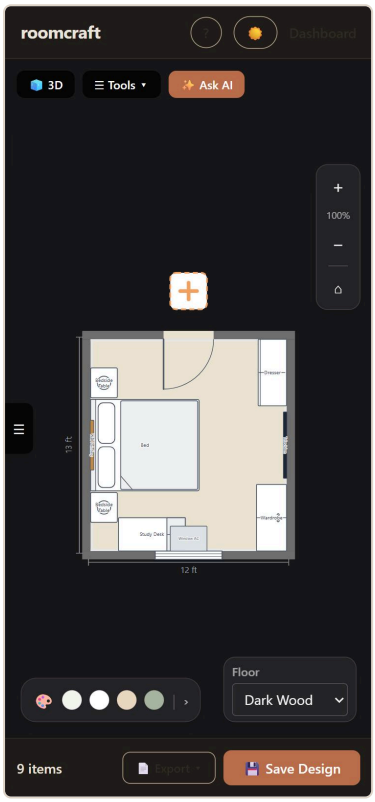


Figure 10.1: 2D plan on a phone



Figure 10.2: 3D view on a phone

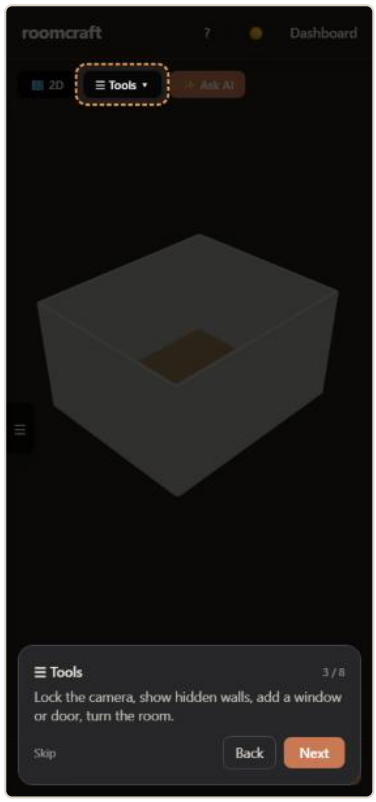


Figure 10.3: Guided tour

11. Security

Area	Measure
Passwords	Stored only as bcrypt hashes (cost 10).
Sessions	Signed JWTs with an expiry; every protected route checks the token.
Account verification	Email one-time codes that expire and work only once; checking them is rate-limited.
Brute force	Rate limits: 10 logins, 5 code requests and 10 code checks per 15 minutes; 10 photo uploads per hour; 12 AI requests per minute; 200 requests per 15 minutes overall.
Secrets	Database password, JWT secret and Gemini key live in server-side `.env` files, excluded from version control; the Gemini key is a free-tier key with no billing attached.
Data access	Every design query is filtered by the logged-in user's id, so users can only see and change their own designs.
Uploads	Profile photos are size-limited and type-checked.
Transport	HTTPS in development (needed by WebXR) and in production.

12. Testing and results

306	2.72 m	1–2 cm	14 / 86
automated scanner tests, all passing	measured ceiling vs 2.74 m by tape	typical error on floor lines	room types / furniture types

12.1 Automated tests

The scanner's geometry is covered by 306 Vitest tests in 16 files, including a complete room scan driven by a simulated AR session. Every geometry test uses rooms that are rotated, away from the origin and with the floor below zero, as in a real AR session, so the code cannot rely on convenient coordinates.

12.2 Measurement accuracy

Measurement	Result
Straight lines on an open floor	Within 1–2 cm typically; about 4 cm on long lines
Ceiling height (real room)	2.72 m against 2.74 m by tape
Room size	Within a few centimetres when tracking is steady and the corners are visible
Furniture size from one tap	Standard catalogue sizes (Single, Double, Queen, King…), not measured

12.3 Functional and interface checks

- Rooms of all 14 types were generated and drawn through the PDF export path; an automatic check found no overlapping labels, and the plans were inspected visually.
- The editor, dashboard, scanner page and tutorial were checked at phone width (375 px) and laptop width (1280 px), in both themes, for overlapping controls.
- The AI assistant was tested live for plans and chat actions, including timeouts, a busy model and replies with missing fields.
- End to end: a room scanned on a phone was saved and opened in the laptop editor with its real size, ceiling height, doors and furniture.

13. Limitations and future work

13.1 Limitations

- **Rectangular rooms only.** L-shaped and irregular rooms must be built from several connected rooms.
- **Android only for scanning.** iPhones do not support WebXR AR in Safari.
- **Tracking drift.** Fast turns, plain walls and shiny floors can make the phone's tracking slip, putting a wall tens of centimetres off. The app flags inconsistent walls and suggests a rescan.
- **Furniture sizes** from a single tap are standard sizes, not measured.
- **AI availability** depends on the free tier of the Gemini API.

13.2 Future work

- **Deployment** to the public web: frontend on Vercel, database on Neon (managed PostgreSQL), sign-up codes by Gmail.
- **Better scanning:** drift correction from re-sighting the first corner, and measured furniture sizes.
- **Irregular rooms** from more than four walls.
- **Shopping links** for furniture that matches the catalogue sizes, and cost estimates.
- **Sharing and collaboration:** read-only links and comments on a design.

14. Conclusion

roomcraft shows that room planning can start from the real room rather than an empty box. A phone in a browser measures the room to within centimetres, the editor turns it into a 2D plan and 3D model that enforce physical rules, and an AI assistant proposes complete layouts and changes that the user applies with a tap. Keeping the AI to choosing items while the app does the placing gave reliable results: AI-made rooms follow the same collision and clearance rules as rooms built by hand.

The project combined web development, 3D graphics, augmented reality, computer vision and generative AI in one product that works on phones and laptops alike, with accounts, saved designs and printable floor plans.

References

1. W3C. *WebXR Device API* and *WebXR Hit Test Module*. https://www.w3.org/TR/webxr/
2. Google. *ARCore overview*. https://developers.google.com/ar
3. three.js documentation. https://threejs.org/docs/
4. pmndrs. *React Three Fiber* documentation. https://r3f.docs.pmnd.rs/
5. React documentation. https://react.dev/
6. Google. *Gemini API: structured output*. https://ai.google.dev/gemini-api/docs/structured-output
7. Google. *MediaPipe Object Detector*. https://ai.google.dev/edge/mediapipe/solutions/vision/object_detector
8. PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/
9. Express documentation. https://expressjs.com/
10. IETF RFC 7519. *JSON Web Token (JWT)*.